

Evaluare

Laborator - 40% din nota finală

- Teme/proiecte fără lucrare.
- obligatoriu pentru promovare examen: **nota test laborator ≥ 5**

Proiect mare:

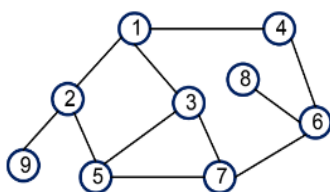
- O clasa a voastra pentru grafuri care va implementa mulți algoritmi.

Modalități de reprezentare a grafurilor

- matrice de adiacență
- liste de adiacență
- listă de muchii

- liste de adiacență

1: 2, 3, 4
2: 1, 9, 5
3: 1, 5, 7
4: 1, 6



5: 2, 3, 7
6: 4, 8, 7
7: 3, 6, 5
8: 6
9: 2

- listă de muchii 1 2

1 3
1 4
2 5
2 9
3 5
3 7
5 7
6 7
6 8
4 6

matricea de adiacență

```
0 1 1 1 0 0 0 0 0
1 0 0 0 1 0 0 0 1
1 0 0 0 1 0 1 0 0
1 0 0 0 0 1 0 0 0
0 1 1 0 0 0 1 0 0
0 0 0 1 0 0 1 1 0
0 0 1 0 1 1 0 0 0
0 0 0 0 0 1 0 0 0
0 1 0 0 0 0 0 0 0
```

FMI – Algoritmica grafurilor, anul 1, Laborator

Parcurgerea grafurilor

Scop: Dat un graf și un vârf de start s , să se determine toate vârfurile accesibile din s (printr-un lanț/drum, după cum graful este neorientat/orientat).

Ideea: pornim din vârful de start s ; dacă știm că un vârful i este accesibil din s și există muchie (arc în caz orientat) de la i la j , atunci și j este accesibil din s .

▪ Parcurgea pe lățime (BF - Breadth First)

Parcurgerea pe lățime BF urmărește vizitarea vârfurilor **în ordinea crescătoare a distanțelor** lor față de s (distanța este dată de numărul de muchii).

Se pornește din vârful de start s , se vizitează vecinii acestuia (vârfurile adiacente cu el), apoi vecinii nevizitați anterior ai acestora, procesul repetându-se până nu mai există vârf cu vecini nevizitați.

Algoritm: Algoritmul va folosi o coadă C în care sunt introduse vârfurile care sunt vizitate și care urmează să fie explorate (în sensul că vor fi cercetați vecinii lor); operațiile de introducere în coadă și eliminare din coadă sunt simbolizate prin $\Rightarrow$, respectiv $\Leftarrow$. Eventuala existență a ciclurilor conduce la necesitatea de a marca vârfurile vizitate. for $i=1, n$

```
vizitat(i)  $\leftarrow$  false  
  
 $C \leftarrow \emptyset$ ;  
  
{se viziteaza varful de start si se introduce in coada}  
  
 $s \Rightarrow C$ ; vizitat( $s$ )  $\leftarrow$  true  
  
while  $C \neq \emptyset$   
  
    {se extrage un varf din coada pentru a fi explorat: se marcheaza ca vizitate varfurile vecine ale acestuia nevizitate anterior si se introduc in coada}  
  
     $i \Leftarrow C$ ; prelucreaza( $i$ );  
    for toți  $j$  vecini ai lui  $i$   
        if not vizitat( $j$ ) then  $j \Rightarrow C$ ; vizitat( $j$ )  $\leftarrow$  true
```

Procedura de prelucrare a unui vârf poate consta, de exemplu, din afișarea informației din vârful respectiv.

FMI – Algoritmica grafurilor, anul 1, Laborator

Muchiile folosite de algoritm pentru a descoperi noi vârfuri accesibile din s formează un **arbore de rădăcină s** . Pentru a reține acest arbore putem folosi legături de tip tată (predecesor). Astfel, vom mai avea un vector $tata$ care se inițializează cu 0. Atunci când vârful j este descoperit ca vecin nevizitat al lui i și introdus în coadă vom atribui lui $tata(j)$ valoarea i (j este fiu al lui i în arbore).

Pentru orice vârf i accesibil din s **lanțul/drumul determinat prin parcurgerea BF de la s la i (din arbore) este minim (ca număr de muchii)**. Putem reconstitui un astfel de

drum folosind legătura $tata$, mergând înapoi din i spre s .

▪ Parcurgerea în adâncime (DF – Depth First)

Pentru parcurgerea în adâncime a componentei conexe a lui s se pornește din acest vârf și se trece mereu la primul dintre vecinii vârfului curent nevizitați anterior, dacă un astfel de vecin există. Dacă toți vecinii vârfului curent au fost deja vizitați se merge înapoi pe drumul de la s la vârful curent până se ajunge la un vârf care mai are vecini nevizitați; se trece la primul dintre aceștia și se reia procedeul.

Procedura recursivă pentru parcurgerea în adâncime (DF) este

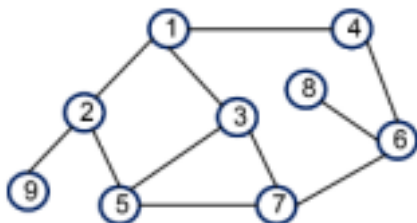
```
procedure DF(i)
  prelucreaza(i);

  vizitat(i) ← true
  for toți j vecini ai lui i
    if not vizitat(j) then DF(j)
```

Vectorul `vizitat` se inițializează cu `false`. Pentru determinarea componentei conexe a lui s procedura se apelează `DF(s)`. Dacă dorim determinarea tuturor componentelor conexe se apelează procedura pentru fiecare vârf rămas nevizitat.

```
for i=1, n
  if not vizitat(i) then DF(i)
```

Exemplu



BF: 1, 2, 3, 4, 5, 9, 7, 6, 8

DF: 1, 2, 5, 3, 7, 6, 4, 8, 9

Am început discuția despre noduri critice/puncte critice/componente biconexe/tare conexe:

Deadline la laboratorul 3:

- 1) <https://infoarena.ro/problema/bfs>
- 2) <https://infoarena.ro/problema/dfs>

Vom mai discuta si la laborul 2 dar deadline e la 3

- 3) <https://infoarena.ro/problema/biconex>
- 4) <https://infoarena.ro/problema/ctc>
- 5) [Havel Hakimi](#) (nu exista problema pe infoarena, prin urmare trebuie sa cititi voi un sir de

n numere si sa spuneti da sau nu)

6) Muchie critica (<https://leetcode.com/problems/critical-connections-in-a-network/>)

Probleme suplimentare pentru bonus:

<http://www.infoarena.ro/problema/rj>

<http://www.infoarena.ro/problema/graf>